Full-Stack Web Development

with React, Next.js & MongoDB

React
Frontend

Next.js
Backend & API

MongoDB
Database

Frontend (React)



- Modern UI with React
- Component-based architecture
- State management & routing
- Responsive design

Backend (Next.js)



- API Routes & Server Actions
- Authentication (Auth.js)
- Data fetching & caching
- File uploads & middleware

Database (MongoDB)



- NoSQL database
- Mongoose ODM
- Data modeling & relationships
- Scalable & cloud-ready (Atlas)

Fast Development | Secure & Scalable | Cloud Deployment (Vercel + Atlas) | Modern Stack

1 Web Fundamentals

The basics of the web



- HTML
- CSS
- JavaScript
- DOM
- APIs
- HTTP/HTTPS

2 React Introduction

A powerful JavaScript library for building user interfaces

- Component-Based
- Declarative UI
- Virtual DOM
- Rich Ecosystem



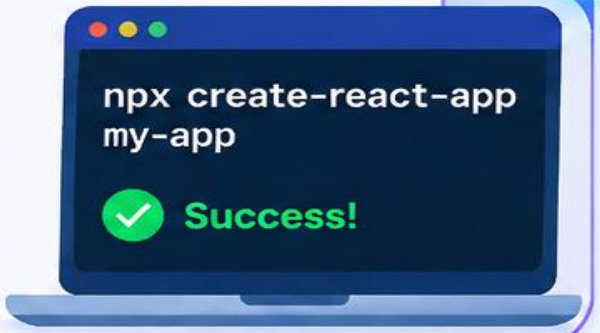
React

Build Modern Web Applications

3 React Getting Started

Set up and build your first React app

1. Install Node.js
2. Create a React App (with Vite or CRA)
3. Explore the Project Structure
4. Run the App



4 What is ES6?

Modern JavaScript Features

- let and const
- Arrow Functions
- Template Literals
- Destructuring
- Modules (import/export)
- Classes





Learn React • Build Amazing Apps • Grow Your Skills

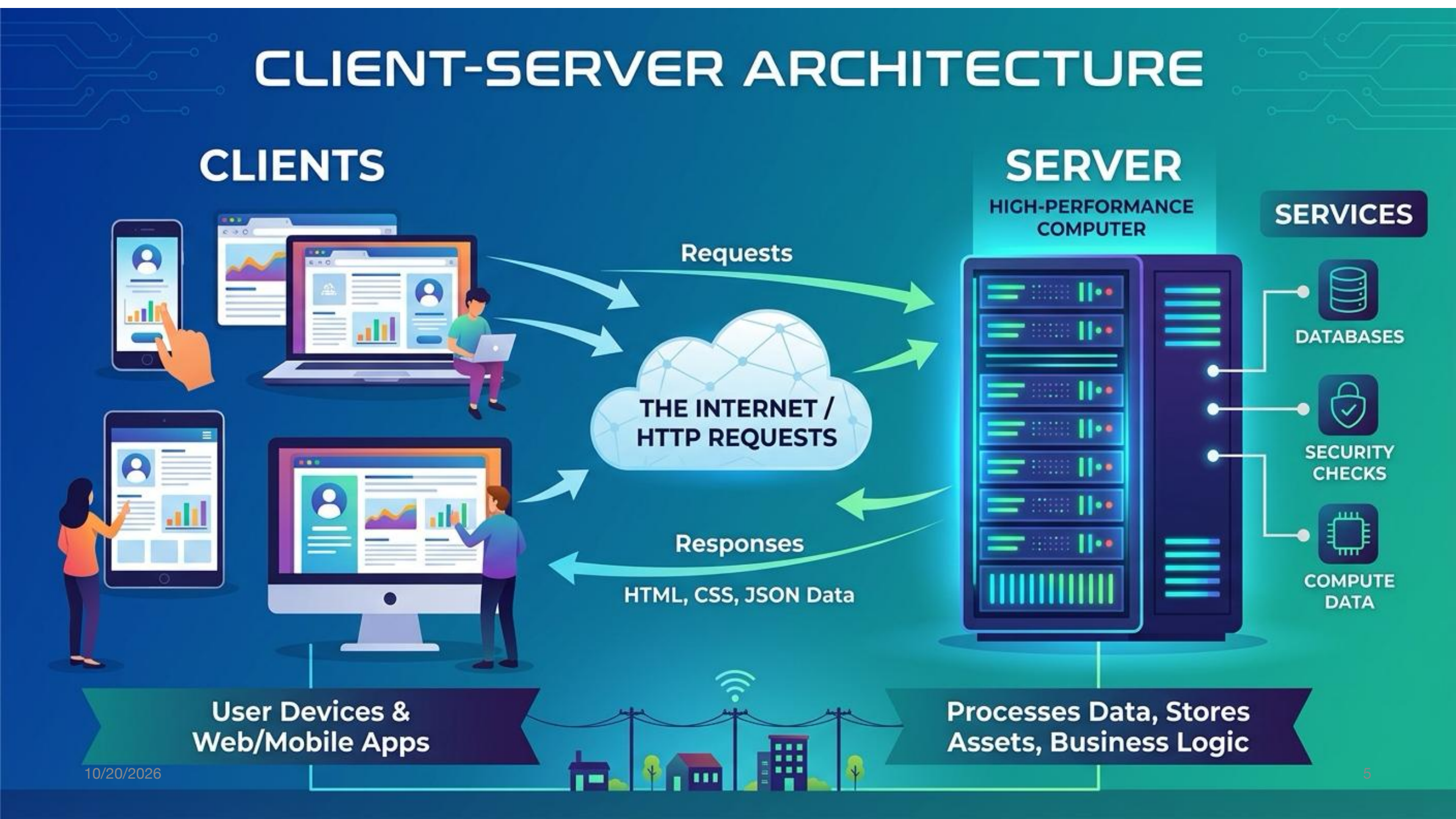
Web Fundamentals

❖ **How the Web Works:**

- **Client-Server Architecture:** Browsers (clients) send requests; remote web servers process requests and return resources.
- **HTTP/HTTPS Protocols:** Stateless request-response communications using standardized methods (GET, POST) and status codes (200, 404, 500).
- **DNS Resolution:** Translating human-readable domain names (e.g., google.com) into numerical IP addresses (142.250.190.46).
- **Request-Response Lifecycle:** Establishes TCP/TLS connections, sends HTTP headers/payloads, and parses returned assets into rendered pages.

Client-Server Architecture

- The modern web relies fundamentally on a distributed client-server model.
- The client is typically a web browser (Chrome, Firefox, Safari) or a mobile application running on a user's device.
- The server is a high-performance computer hosted locally or in the cloud (AWS, Azure, Vercel) that stores website assets, business logic, and databases.
- The client's job is to present the user interface and capture inputs, while the server's job is to compute data, run security checks, and send back requested files.


CLIENT-SERVER ARCHITECTURE
CLIENTS
SERVER
HIGH-PERFORMANCE COMPUTER
SERVICES
Requests
THE INTERNET / HTTP REQUESTS
DATABASES
SECURITY CHECKS
COMPUTE DATA
Responses
HTML, CSS, JSON Data
User Devices & Web/Mobile Apps
Processes Data, Stores Assets, Business Logic

HTTP/HTTPS Protocols

- HTTP (Hypertext Transfer Protocol) and HTTPS (HTTP Secure, encrypted via TLS/SSL) define the standardized language clients and servers use to talk to each other.
- Communication is stateless, meaning each request is independent and has no memory of previous interactions unless explicitly managed by sessions or tokens.
- Common HTTP Methods:
  - ✓ **GET**: Fetch data from the server (e.g., loading a profile page).
  - ✓ **POST**: Send new data to the server (e.g., submitting a registration form).
  - ✓ **PUT / PATCH**: Update existing data on the server.
  - ✓ **DELETE**: Remove data from the server.

HTTP / HTTPS PROTOCOLS

CLIENT-SERVER COMMUNICATION



HTTPS
Encrypted
(Secure, via TLS/SSL)

COMMON HTTP METHODS

GET
Fetch Data
(e.g., load page)

POST
Send New Data
(e.g., submit registration)

PUT/PATCH
Update Existing Data

DELETE
Remove Data

KEY STATUS CODES

200 — **OK:** Request succeeded.

301 — **MOVED PERMANENTLY:** Redirecting to a new URL.

400 — **BAD REQUEST:** Client sent invalid data.

401/403 — **UNAUTHORIZED / FORBIDDEN:** Authentication or permission failure.

404 — **NOT FOUND:** The requested resource does not exist.

500 — **INTERNAL SERVER ERROR:** The server crashed or encountered a bug.

REACT: BUILDING THE FUTURE OF UI

Introduce an immersive developer's console and launchpad

COMPONENT-BASED ARCHITECTURE

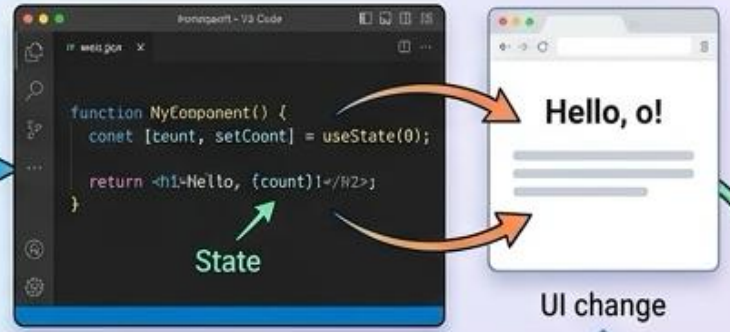


Reusable UIs

Lego-like building

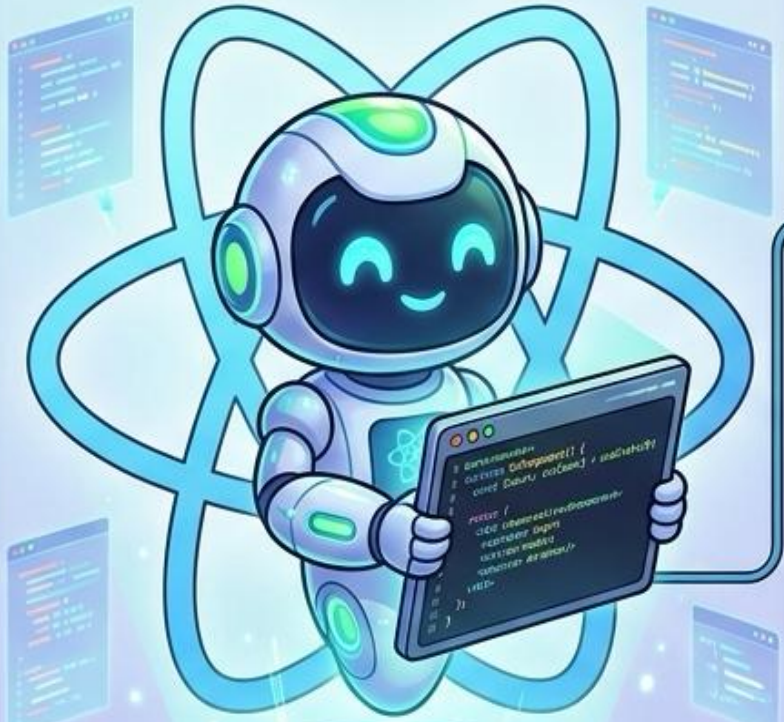
Simple to manage

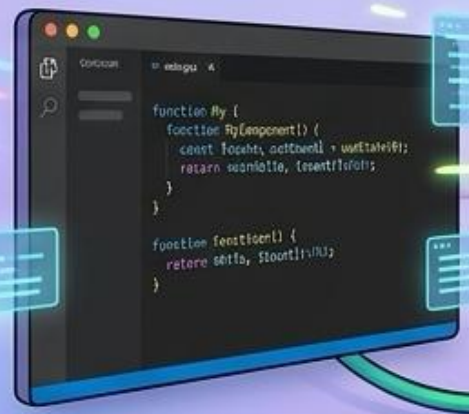
DECLARE & UPDATE (JSX & STATE)



UI change

UI reacts to data changes.





MODERN TOOLS



START NEW PROJECT

Efficient and fast development.

React Introduction

What is React?

- React is a **front-end** JavaScript library.
- React was developed by the **Facebook** Software Engineer Jordan Walke.
- React is also known as **React.js** or **ReactJS**.
- React is a tool for building **UI components**.



How does React Work?

- User interacts with the application.
- State changes.
- React creates a new Virtual DOM.
- React compares old and new Virtual DOM (Diffing).
- React updates only the changed elements in the Real DOM.
- The UI updates efficiently without reloading the entire page.

HOW DOES REACT WORK?

STEP 1: DEFINE COMPONENTS





Building blocks of UI

STEP 2: DECLARE STATE & UPDATES



Initial State:
Counter = 0

```
const [count, setCount] = useState(0);

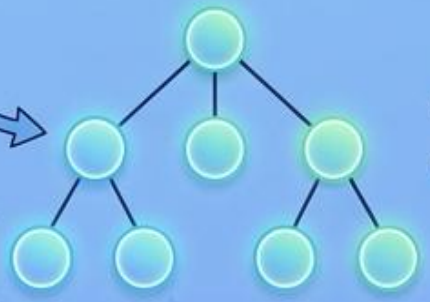
setCount(count + 1)
```

State Update:
Counter = 1

UI reacts to data changes

STEP 3: VIRTUAL DOM & RECONCILIATION

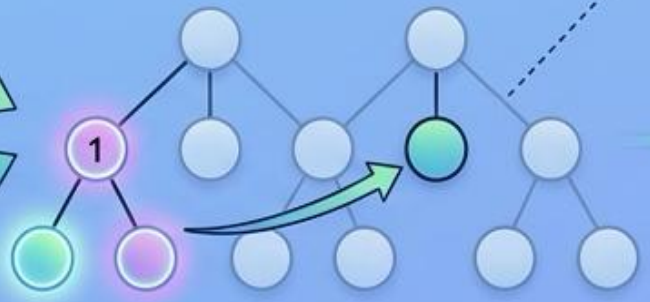
Virtual DOM
(Blueprint)



Fast, in-memory representation

Compare
(Diffing)

Real DOM
(The Browser)



Efficient, targeted update



Diffing algorithm

STEP 4: RENDER & REFRESH



Final, rendered UI to the screen

React.JS History

From a Facebook project to a global JavaScript library

Same Vision
Better Tools
Bigger Impact

2011 — Initial Idea

Facebook started working on React to solve the challenges of building large, complex web applications.

2013 — Open Sourced

React was released as an open-source project at JavaScript Conf 2013 with a focus on UI components and performance.

2015 — React 0.14

Introduced Server-Side Rendering (SSR) and better component structure, making React more flexible and powerful.

2017 — React 16

Brought Fiber (a new rendering engine) for better performance and introduced Error Boundaries for safer apps.

2019 — React Hooks

Made it easier to use state and other React features in functional components.

2022+ — React 18 and Beyond

Introduced concurrent features, automatic batching, and improved performance. Now it powers modern web, mobile, and full-stack apps.



From a small internal project to a global ecosystem

React.JS has grown from a Facebook experiment in 2011 to one of the most popular JavaScript libraries, used by millions of developers around the world.

React Getting Started

Step 1: Install Node.js

React requires Node.js to run and manage packages.

1. Visit the Node.js website.
2. Download the **LTS (Long-Term Support)** version.
3. Install Node.js using the default settings.

After installation, verify:

Bash

```
node -v
```

Cont. ...

Step 2: Install Visual Studio Code

Visual Studio Code (VS Code) is a popular code editor for React development.

1. Download VS Code.
2. Install it.
3. Launch the application.

Recommended Extensions:

- ES7+ React/Redux Snippets
- Prettier
- Live Server
- Auto Rename Tag

Cont. ...

Step 3: Create a React Project Using Vite

Open Terminal (Mac/Linux) or Command Prompt (Windows).

Navigate to your desired folder:

```bash
cd Desktop/Projects
```

Create a React project:

```bash
npm create vite@latest my-react-app
```

```
npm create vite@latest my-react-app -- --template react
```

You will be asked:

```
Project Name: my-react-app
Framework: React
Variant: JavaScript
```

Cont. ...

What this command means (step by step):

Part	Meaning
`npm create`	Tells npm to run a package's create/scaffolding tool
`vite@latest`	Uses the **latest version** of Vite (a very fast build tool)
`my-app`	The name of the folder/project that will be created
`--`	Separates npm options from the options that belong to Vite
`--template react`	Tells Vite to create a project using the**React** template

Bash

```
npm create vite@latest my-app -- --template react
```

In simple words:

This command creates a **new React project** called `my-app` using **Vite** (instead of the older Create React App).

What happens after you run it:

1. It creates a folder named `my-app`
2. Sets up a modern React + Vite project structure
3. Installs the basic files (like `package.json`, `src/App.jsx`, etc.)

Cont. ...

Step 4: Move into the Project Folder

Bash

```
cd my-react-app
```

Step 5: Install Project Dependencies

Install all required packages:

Bash

```
npm install
```

This creates the `node_modules` folder and downloads all required libraries.

Step 6: Open the Project in VS Code

Bash

```
code .
```

Step 7: Run the React Application

Start the development server:

Bash

```
npm run dev
```

Example Output:

Bash

```
VITE v8.x.x ready in 300 ms

→ Local: http://localhost:5173/
```

Cont. ...

```
[→  React_Pro mkdir React_App
[→  React_Pro cd React_App
[→  React_App node -v
v24.21.0
[→  React_App npm -v
11.19.0
[→  React_App npm create vite@latest React_Demo

> npx
> 'create-vite' React_Demo

│
◇  Package name:
│  react-demo
│
◇  Select a framework:
│  Vanilla
│
◇  Select a variant:
│  TypeScript
│
◇  Install with npm and start now?
│  Yes
```

```
│
◇  Scaffolding project in /Users/eaii/Desktop/Projects
│
◇  Installing dependencies with npm...

added 16 packages, and audited 17 packages in 22s

8 packages are looking for funding
  run `npm fund` for details

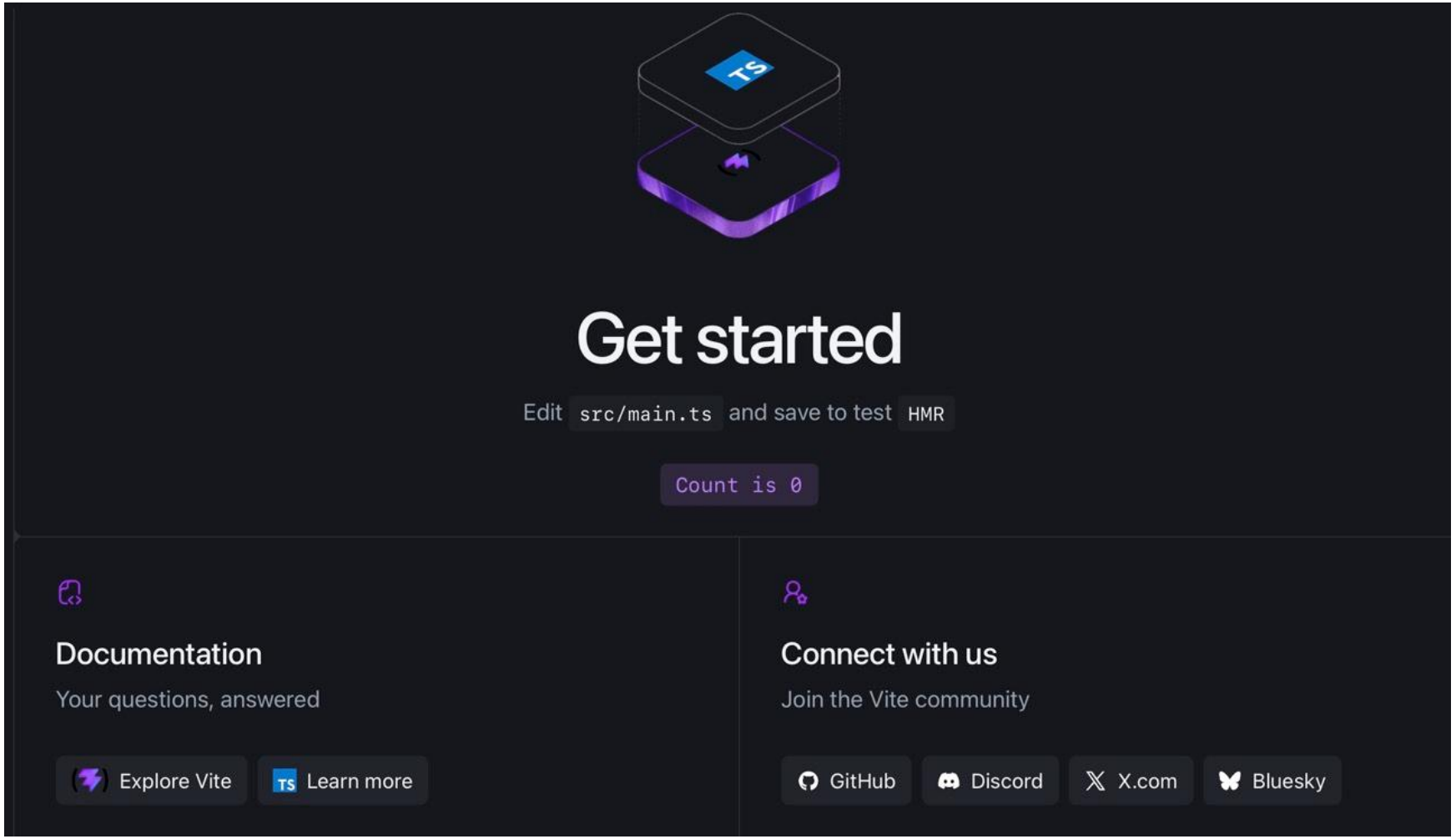
found 0 vulnerabilities
│
◇  Starting dev server...

> react-demo@0.0.0 dev
> vite

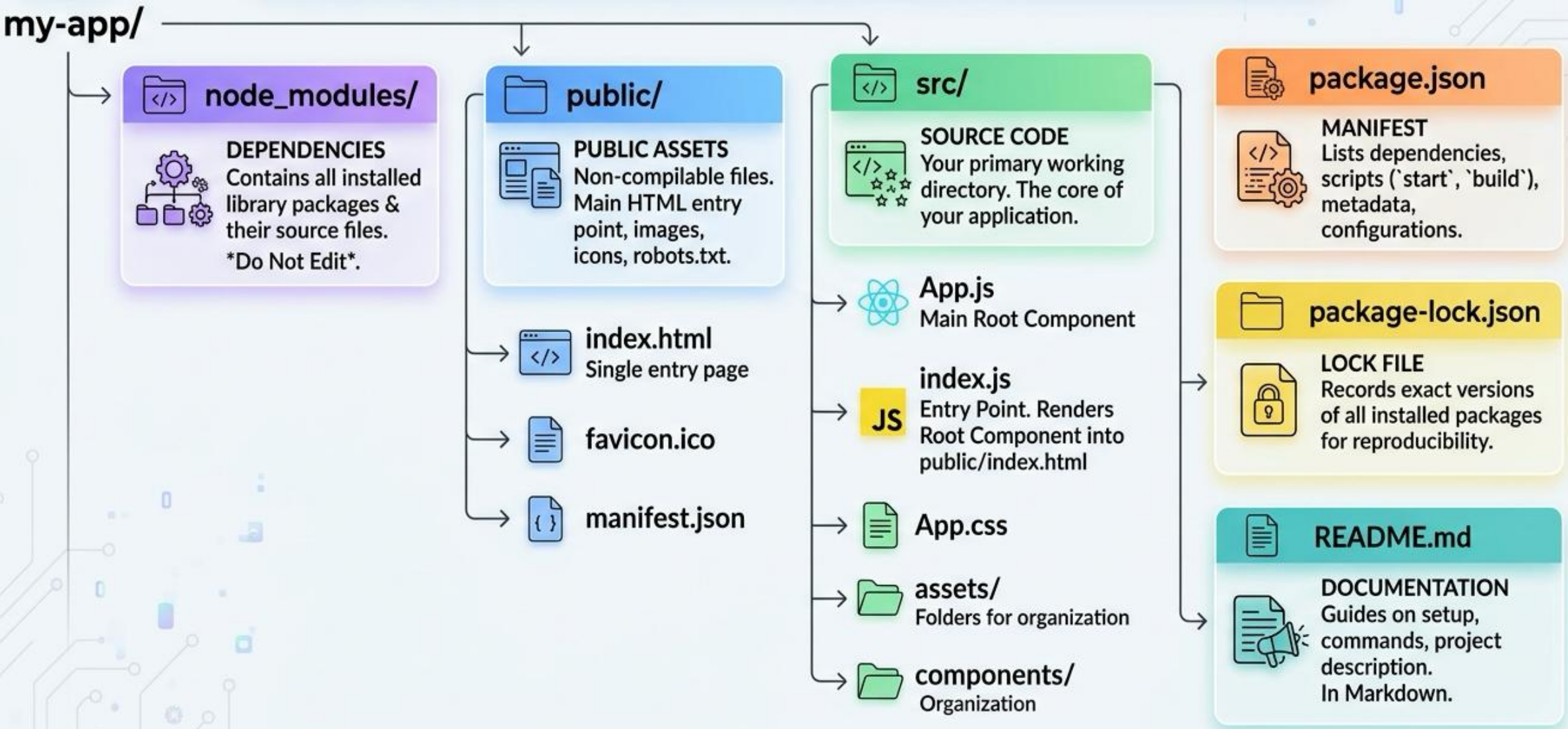

  VITE v8.3.2  ready in 689 ms

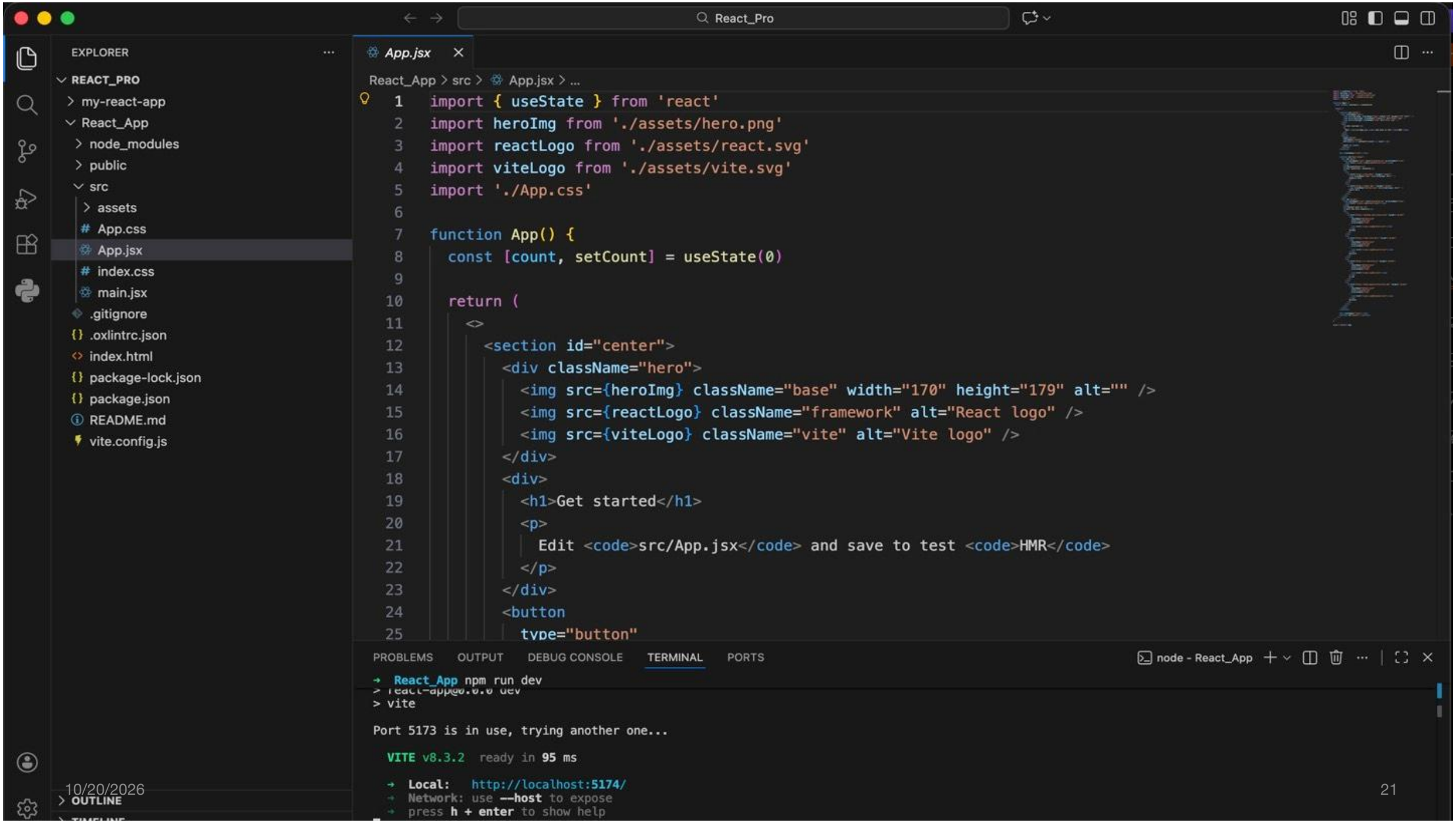
  →  Local:   http://localhost:5173/
  →  Network: use --host to expose
  →  press h + enter to show help
```

First Landing

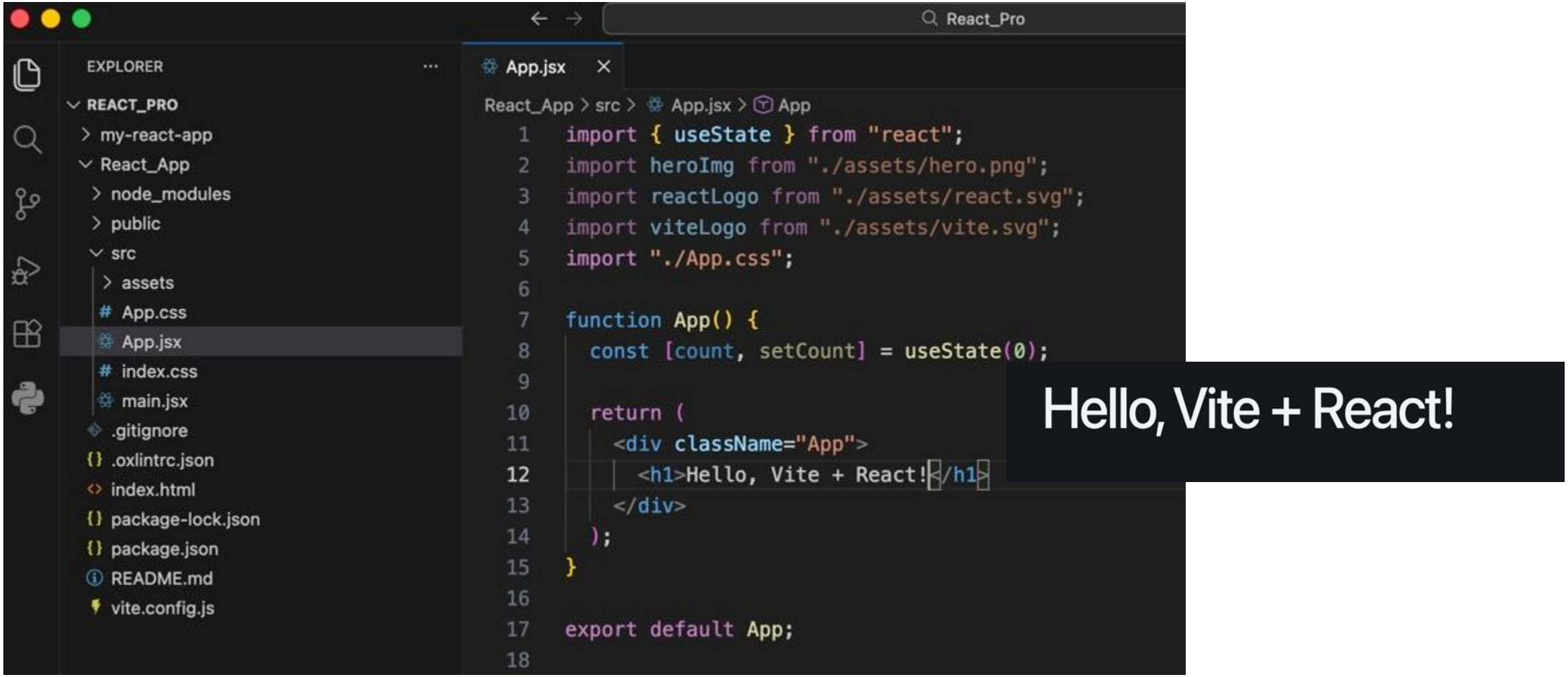


REACT PROJECT STRUCTURE (Fresh Installation)




React_Pro
EXPLORER
REACT_PRO
my-react-app
React_App
node_modules
public
src
assets
App.css
App.jsx
index.css
main.jsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
vite.config.js
App.jsx
React_App > src > App.jsx > ...
1 import { useState } from 'react'
2 import heroImg from './assets/hero.png'
3 import reactLogo from './assets/react.svg'
4 import viteLogo from './assets/vite.svg'
5 import './App.css'
6
7 function App() {
8 const [count, setCount] = useState(0)
9
10 return (
11 <>
12 <section id="center">
13
14
15
16
17
18
19 <h1>Get started</h1>
20
21 Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
22
23
24 <button
25 type="button"
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
node - React_App
→ React_App npm run dev
> vite
Port 5173 is in use, trying another one...
VITE v8.3.2 ready in 95 ms
→ Local: http://localhost:5174/
→ Network: use --host to expose
→ press h + enter to show help
OUTLINE

Modify the React App



React Render HTML

❖**What is React Render HTML?**

- React rendering is the process of displaying HTML content on a web page from React components. React takes the component's JSX code and converts it into HTML that the browser can understand and display.

❖**Why is Rendering Important?**

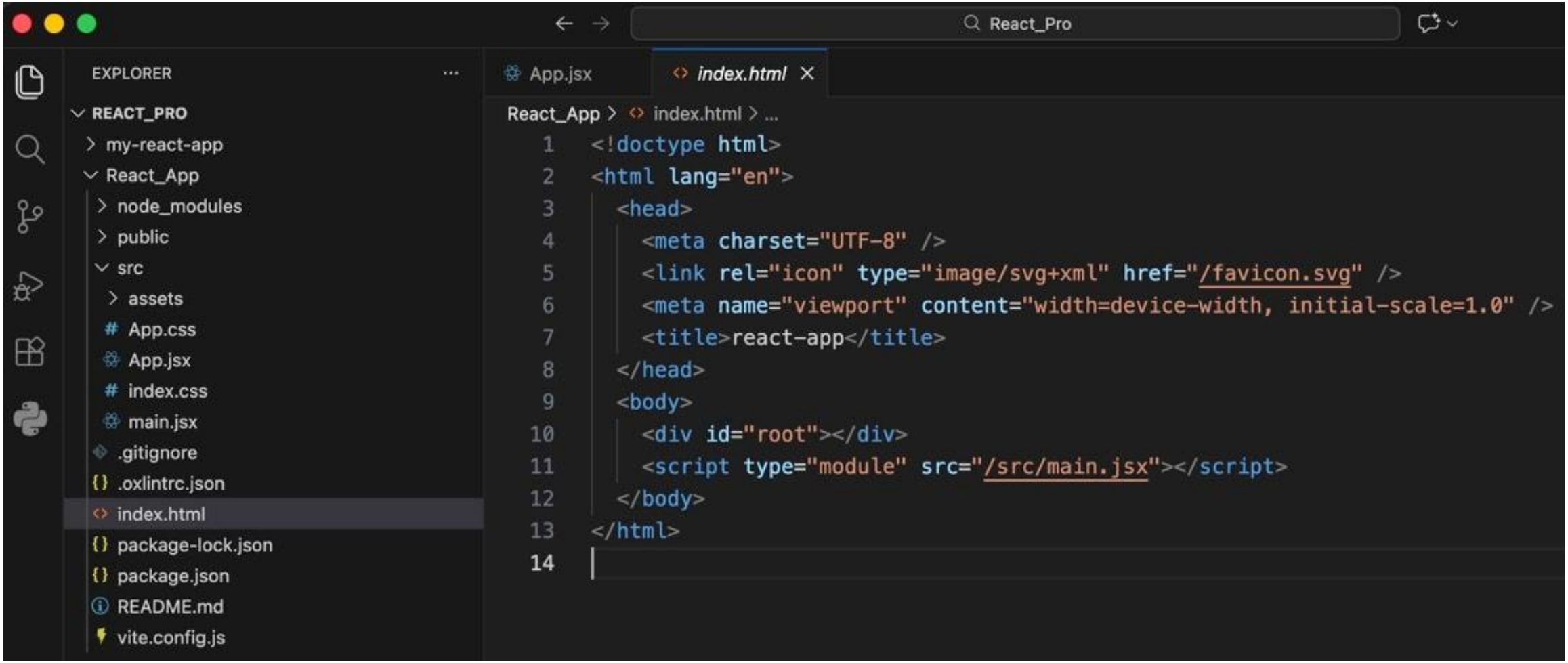
- ✓Displays content to users.
- ✓Updates the user interface when data changes.
- ✓Creates interactive and dynamic web applications.
- ✓Improves performance by updating only the parts of the page that change.

How Does React Render HTML?

- A React component returns JSX.
- React converts JSX into HTML elements.
- React creates a Virtual DOM representation.
- React compares changes in the Virtual DOM.
- Only the changed parts are updated in the browser (Real DOM).

❖**Container**: The container is an HTML element where React displays (renders) the application. It acts as the root location where all React components are loaded and shown in the browser.

Cont. ...


React_Pro
EXPLORER
REACT_PRO
my-react-app
React_App
node_modules
public
src
assets
App.css
App.jsx
index.css
main.jsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
vite.config.js
App.jsx
index.html
React_App > index.html > ...
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7 <title>react-app</title>
8 </head>
9 <body>
10
11 <script type="module" src="/src/main.jsx"></script>
12 </body>
13 </html>
14

The createRoot Function

❖ **What is createRoot()?**

- The createRoot() function is a React method used to create a root where a React application can be rendered. It connects React to the HTML container and serves as the starting point of the React application.

❖ **Why is createRoot() Needed?**

✓ Creates a React root for rendering components.

✓ Connects React with the HTML container ( ).

✓ Enables React's modern rendering features and improved performance.

✓ Manages updates to the user interface efficiently.

Cont. ...

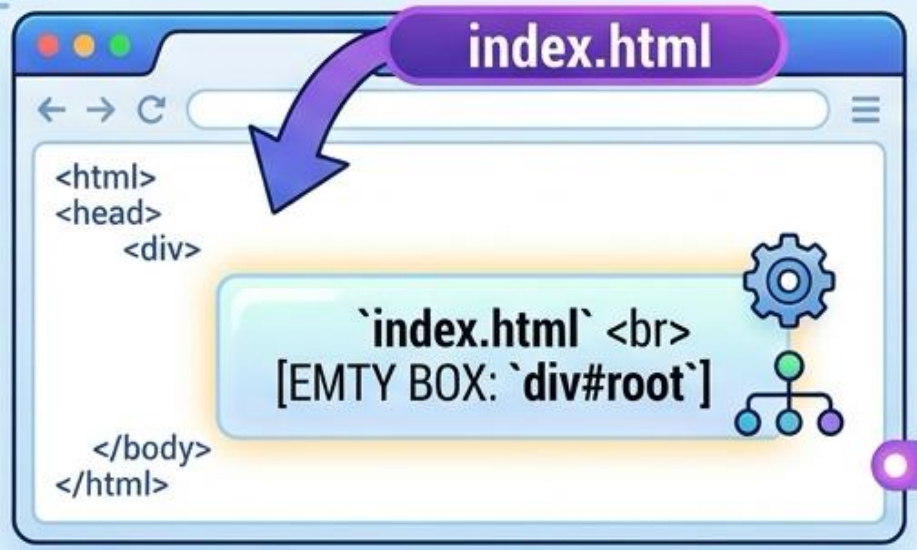
App.jsx | main.jsx

React_App > src > main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

REACT APP INTEGRATION FLOW: HOW IT COMES TOGETHER.



main.jsx finds that div



`main.jsx`
 [Puts React App into empty box]

main.jsx renders <App /> inside it





VISUAL SUMMARY

File	File Name	Role	Role Description
	index.html		EMPTY BOX (`#root`)
	main.jsx		Puts React App into empty box
	App.jsx		The ACTUAL CONTENT users see

What is ES6?

❖ **What is React ES6?**

- ES6 (ECMAScript 2015) is a modern version of JavaScript that introduces new features and syntax to make **code simpler**, **cleaner**, and **easier** to maintain. React applications heavily use ES6 features for **component development** and **state management**.

❖ **Why is ES6 Important in React?**

- ✓ Makes React code more readable and concise.
- ✓ Reduces the amount of code developers need to write.
- ✓ Provides modern features used throughout React applications.
- ✓ Improves code organization and maintainability.

Cont. ...

- React uses ES6, and you should be familiar with some of the new features like:
  - ✓Classes
  - ✓Arrow Functions
  - ✓Variables (let, const, var)
  - ✓Array Methods like .map()
  - ✓Destructuring
  - ✓Modules
  - ✓Ternary Operator
  - ✓Spread Operator

React ES6 Classes

❖ **What are ES6 Classes?**

- ES6 Classes are a feature introduced in JavaScript ES6 that provide a structured way to create objects and components. Before React Hooks were introduced, class components were commonly used to manage state and lifecycle methods in React applications.

❖ **Why are ES6 Classes Important?**

✓ Organize code in a structured and reusable way.

✓ Allow creation of React class components.

✓ Support state management and lifecycle methods.

✓ Improve code readability and maintainability.

Cont. ...

Basic ES6 Class Syntax

JavaScript

```
class Person {
  constructor(name) {
    this.name = name;
  }

  display() {
    return this.name;
  }
}
```

In this example:

- `class` creates a new class.
- `constructor()` initializes object properties.
- Methods define the behavior of the class.

React Class Component Example

JavaScript

```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>Welcome to React</h1>;
  }
}

export default Welcome;
```

This class component displays a heading in the browser.

Methods in Classes

❖**What are Methods in Classes?**

- Methods are functions defined inside a class. They describe the actions or behaviors that an object or class can perform. In React class components, methods are often used **to handle events**, **update data**, and **perform specific tasks**.

❖**Why are Methods Important?**

- Define the behavior of a class.
- Organize related functionality inside a component.
- Handle user interactions such as button clicks.
- Improve code reusability and readability.

Cont. ...

Methods in a React Class Component

JavaScript

```javascript
import React, { Component } from "react";

class Welcome extends Component {
  showMessage() {
    alert("Welcome to React!");
  }

  render() {
    return (
      <button onClick={() => this.showMessage()}>
        Click Me
      </button>
    );
  }
}

export default Welcome;
```

Here, `showMessage()` is a method that is called when the button is clicked.

Basic Class Method Example

JavaScript

```javascript
class Person {
  greet() {
    return "Hello!";
  }
}

const person = new Person();
console.log(person.greet());
```

Class Inheritance

❖**What is Class Inheritance?**

- Class Inheritance is an ES6 feature that allows one class to inherit properties and methods from another class. This helps developers reuse code and create relationships between classes.
- In React, class components use inheritance by extending **React.Component**.

❖**Why is Inheritance Important?**

- Promotes code reusability.
- Reduces duplicate code.
- Allows one class to use features of another class.
- Makes applications easier to maintain and extend.

Cont. ...

Basic Inheritance Example

JavaScript

```
class Person {
  greet() {
    return "Hello!";
  }
}

class Student extends Person {
}

const student = new Student();

console.log(student.greet());
```

In this example:

- `Person` is the parent (base) class.
- `Student` is the child (derived) class.
- `Student` automatically inherits the `greet()` method.

React Example

JavaScript

```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>Welcome to React</h1>;
  }
}

export default Welcome;
```

Here:

- `Component` is a class provided by React.
- `Welcome` inherits from `Component`.
- As a result, `Welcome` can use React features such as `render()`, `props`, and `state`.

React ES6 Arrow Functions

❖**What are Arrow Functions?**

- Arrow Functions are a shorter and more modern way of writing JavaScript functions introduced in ES6.
- They make code more concise and are widely used in React components, event handlers, and state updates.

❖**Why are Arrow Functions Important?**

✓Reduce the amount of code to write.

✓Make functions easier to read.

✓Commonly used in React components and event handling.

✓Automatically inherit this from the surrounding scope.

Cont. ...

Traditional Function

```javascript
function greet() {
  return "Hello React";
}
```

Shorter Arrow Function Syntax

If the function contains only one statement, the `return` keyword and curly braces can be omitted.

```javascript
const greet = () => "Hello React";
```

Arrow Function

```javascript
const greet = () => {
  return "Hello React";
};
```

Both functions produce the same result, but the arrow function is shorter.

Arrow Function in React Component

```javascript
const Welcome = () => {
  return <h1>Welcome to React</h1>;
};

export default Welcome;
```

This is a functional React component written using an arrow function.

Cont. ...

Arrow Function with Parameters

JavaScript

```
const greet = (name) => {
  return "Hello " + name;
};
```

Example:

JavaScript

```
greet("John");
```

Output:

```
Hello John
```

Arrow Functions for Event Handling

JavaScript

```
function App() {
  const showMessage = () => {
    alert("Button Clicked");
  };

  return (
    <button onClick={showMessage}>
      Click Me
    </button>
  );
}
```

In this example, an arrow function is used to handle a button click event.

React ES6 Variables

❖ **What are ES6 Variables?**

- ES6 Variables are used to store data in JavaScript. ES6 introduced two new keywords, **let** and **const**, which provide better control and flexibility compared to the older **var** keyword.
- React applications commonly use **let** and **const** for declaring variables.

❖ **Why are ES6 Variables Important?**

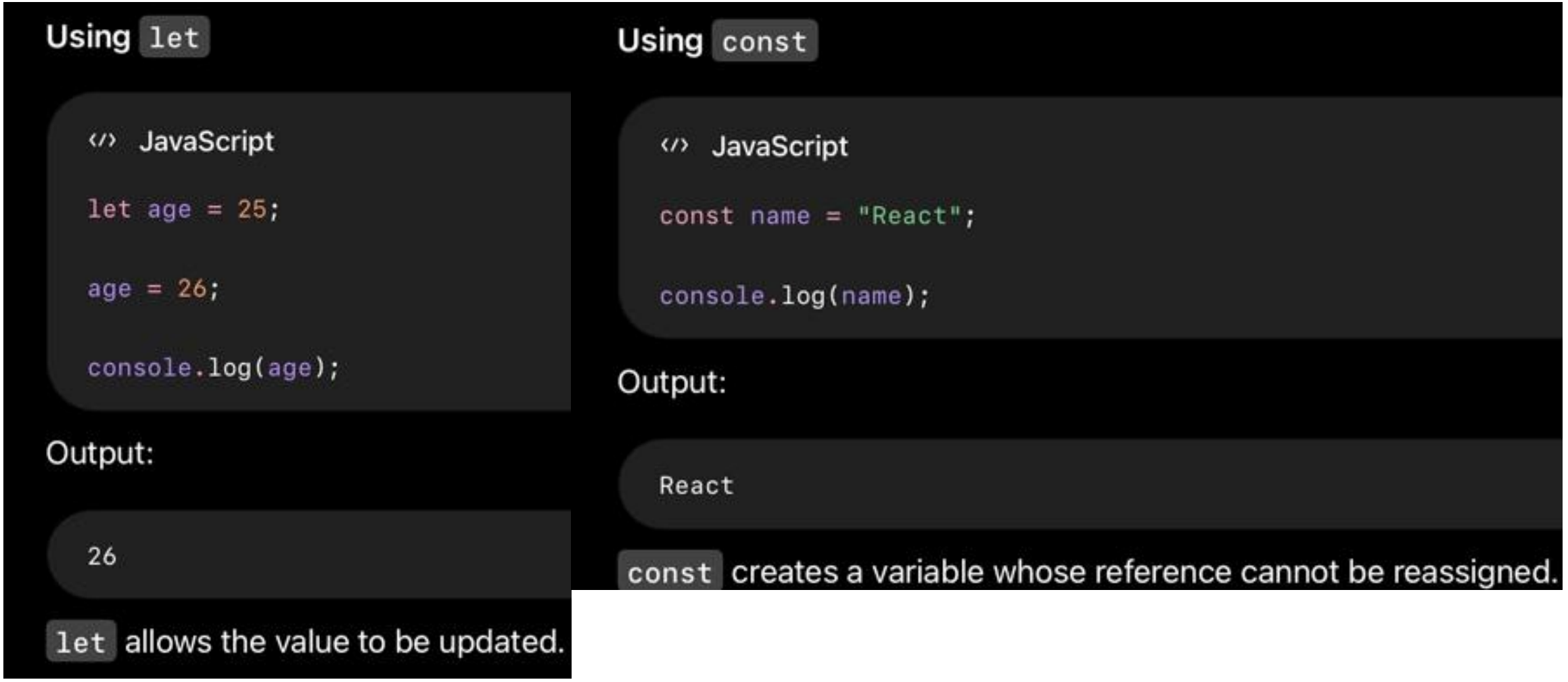
- ✓ Store data used in React applications.
- ✓ Improve code readability and maintainability.
- ✓ Help prevent accidental changes to values.
- ✓ Provide block-level scope for better control.

Cont. ...

Types of ES6 Variables

Keyword	Description
`var`	Older way of declaring variables
`let`	Variable value can be changed
`const`	Variable value cannot be reassigned

Cont. ...


Using let
JavaScript
let age = 25;
age = 26;
console.log(age);
Output:
26
let allows the value to be updated.
Using const
JavaScript
const name = "React";
console.log(name);
Output:
React
const creates a variable whose reference cannot be reassigned.

Cont. ...

Variables in React Components

JavaScript

```
function App() {
  const title = "Welcome to React";

  return <h1>{title}</h1>;
}

export default App;
```

In this example:

- `title` stores text.
- React displays the value inside JSX using `{}`.

Example with Multiple Variables

JavaScript

```
function App() {
  const name = "John";
  let age = 25;

  return (
    <div>
      <h1>{name}</h1>
      <p>Age: {age}</p>
    </div>
  );
}

export default App;
```

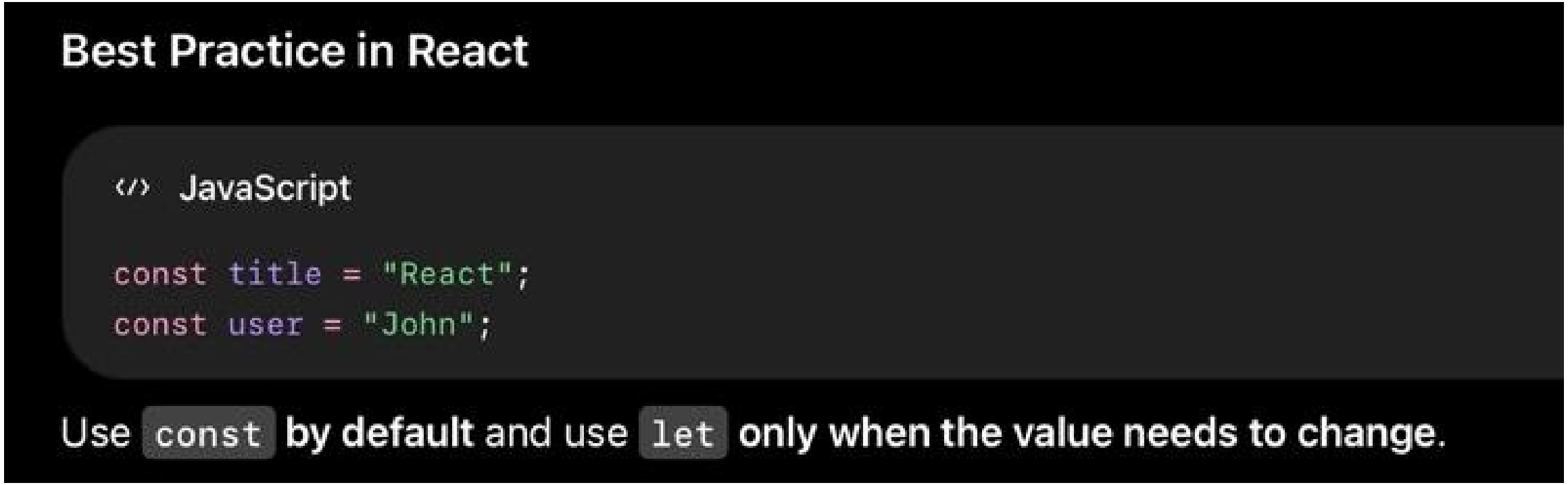
React displays the values stored in the variables.

Cont. ...

`let` vs `const`

`let`	`const`
Value can be changed	Value cannot be reassigned
Used for changing data	Used for fixed values
Block scoped	Block scoped
Less commonly used in React	Most commonly used in React

Cont. ...



Key Point: ES6 Variables (let and const) are used to store data in React applications. const is preferred for values that do not change, while let is used when a value needs to be updated. These variables can be displayed inside JSX and rendered in the browser.

JavaScript Array map()

❖**What is map()?**

- The map() method is a JavaScript array function used to create a new array by applying a function to each item in an existing array.
- In React, map() is commonly used to render lists of data as HTML elements.

❖**Why is map() Important in React?**

✓Displays multiple items from an array.

✓Avoids writing repetitive HTML.

✓Makes list rendering dynamic.

✓Commonly used to display users, products, courses, or other data.

Cont. ...

Basic JavaScript Example

JavaScript

```javascript
const numbers = [1, 2, 3, 4];

const doubled = numbers.map((number) => {
  return number * 2;
});

console.log(doubled);
```

Output:

```
[2, 4, 6, 8]
```

`map()` processes each item and returns a **new array**.

Using `map()` in React

JavaScript

```javascript
function App() {
  const students = ["Abel", "Sara", "John"];

  return (
    <ul>
      {students.map((student) => (
        <li>{student}</li>
      ))}
    </ul>
  );
}

export default App;
```

The browser displays:

- Abel
- Sara
- John

Cont. ...

`map()` with Objects

JavaScript

```
const students = [
  { id: 1, name: "Abel" },
  { id: 2, name: "Sara" },
  { id: 3, name: "John" }
];

function App() {
  return (
    <div>
      {students.map((student) => (
        <p key={student.id}>
          {student.name}
        </p>
      ))}
    </div>
  );
}
```

Here, `map()` converts each student object into a `<p>` element.

Key Point

- **map()** is a JavaScript array method that processes each item in an array and returns a **new array**.
- In React, it is commonly used to dynamically render **lists of elements** from data.

React ES6 Destructuring

❖ **What is Destructuring?**

- Destructuring is an ES6 feature that allows you to extract values from **arrays** or **objects** and store them in variables using a shorter and cleaner syntax.
- In React, destructuring is commonly used with **props**, **state**, and **data objects**.

❖ **Why is Destructuring Important in React?**

- Makes code shorter and easier to read.
- Provides easy access to object or array values.
- Commonly used with React props and state.
- Reduces repetitive code.

Cont. ...

Object Destructuring

JavaScript

```
const student = {
  name: "Abel",
  age: 22
};

const { name, age } = student;

console.log(name);
console.log(age);
```

Output:

```
Abel
22
```

Destructuring Props in React

Without destructuring:

JavaScript

```
function Student(props) {
  return <h2>{props.name}</h2>;
}
```

With destructuring:

JavaScript

```
function Student({ name }) {
  return <h2>{name}</h2>;
}
```

The second approach is shorter and easier to read.

Array Destructuring

JavaScript

```
const colors = ["Red", "Green", "Blue"];

const [first, second, third] = colors;

console.log(first);
console.log(second);
console.log(third);
```

Output:

```
Red
Green
Blue
```

Cont. ...

Destructuring with `useState`

A very common example in React is:

```javascript
const [count, setCount] = useState(0);
```

Here:

- `count` → current state value
- `setCount` → function used to update the value

The two values are extracted from the array returned by `useState()`.

How Does Destructuring Work?

```
Object / Array
      ↓
Destructuring
      ↓
Extract Values
      ↓
Store in Variables
      ↓
Use in React Component
```

React ES6 Spread Operator

❖ **What is the Spread Operator?**

- The Spread Operator **(...)** is an ES6 feature used to **expand** or **copy** the elements of an array or the properties of an object.
- In React, it is commonly used to copy data, combine objects or arrays, and pass multiple props to a component.

❖ **Why is the Spread Operator Important in React?**

✓ Quickly copies arrays and objects.

✓ Combines multiple arrays or objects.

✓ Makes it easier to work with React props and state.

✓ Helps avoid directly modifying existing data.

Cont. ...

Spread Operator with Arrays

JavaScript

```javascript
const numbers = [1, 2, 3];

const newNumbers = [...numbers, 4, 5];

console.log(newNumbers);
```

Output:

```
[1, 2, 3, 4, 5]
```

The `...numbers` spreads the existing elements into the new array.

Spread Operator with Objects

JavaScript

```javascript
const student = {
  name: "Abel",
  age: 22
};

const updatedStudent = {
  ...student,
  department: "Software Engineering"
};
```

Result:

```
{
  name: "Abel",
  age: 22,
  department: "Software Engineering"
}
```

Cont. ...

Spread Operator with React Props

The spread operator can pass multiple object properties as props.

JavaScript

```
const student = {
  name: "Abel",
  age: 22
};

<Student {...student} />
```

This is equivalent to:

JavaScript

```
<Student
  name={student.name}
  age={student.age}
/>
```

Spread Operator with State

The spread operator is commonly used when updating objects in React state.

JavaScript

```
const [student, setStudent] = useState({
  name: "Abel",
  age: 22
});

setStudent({
  ...student,
  age: 23
});
```

Here, React creates a new object containing the existing properties and updates only the `age`.

React ES6 Modules

❖ **What are ES6 Modules?**

- ES6 Modules allow JavaScript code to be split into separate files and reused across an application. In React, modules are commonly used to organize components, functions, data, and other code into separate files.
- The two main keywords are export and import.

❖ **Why are Modules Important in React?**

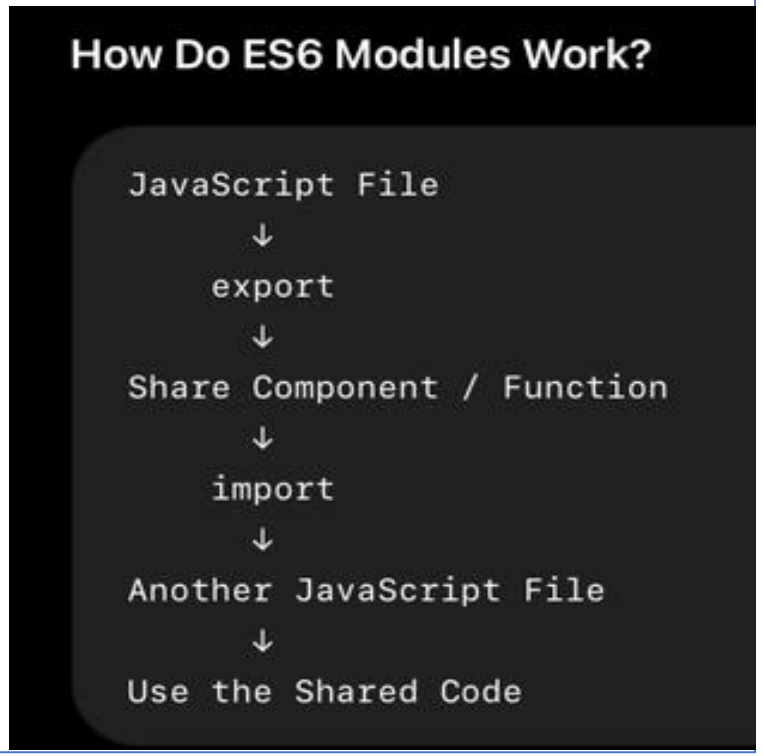
✓ Organize code into separate files.

✓ Make components reusable.

✓ Improve code readability and maintainability.

✓ Prevent large files from becoming difficult to manage.

✓ Allow different parts of an application to share code.



Cont. ...

1. Export a Component

Create `Header.jsx`:

JavaScript

```
function Header() {
  return <h1>Welcome to React</h1>;
}

export default Header;
```

Here, `export default` makes the `Header` component available to other files.

Cont. ...

2. Import the Component

In `App.jsx`:

JavaScript

```
import Header from "./Header";

function App() {
  return (
    <div>
      <Header />
    </div>
  );
}

export default App;
```

React can now use the `Header` component in `App`.

Named Export

A file can also export multiple items:

JavaScript

```
export const name = "Abel";
export const age = 22;
```

Import them using:

JavaScript

```
import { name, age } from "./student";
```

Note: Named exports use `{ }`, while a default export does not.

Cont. ...

Default vs Named Export

Default Export	Named Export
`export default Header`	`export { Header }`
Import without `{ }`	Import with `{ }`
Usually one default export per file	Multiple named exports allowed
`import Header from "./Header"`	`import { Header } from "./Header"`

Key Point

ES6 Modules allow React applications to divide code into separate, reusable files. The `export` keyword shares code, while the `import` keyword brings that code into another file.

React ES6 Ternary Operator

❖What is the Ternary Operator?

- The Ternary Operator (? :) is a short way to write an if...else condition. In React, it is commonly used to conditionally display different content or components.



Cont. ...

Basic JavaScript Example

JavaScript

```
const age = 20;

const result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

Output:

```
Adult
```

The condition `age >= 18` is true, so `"Adult"` is returned.

Ternary Operator in React

JavaScript

```
function App() {
  const isLoggedIn = true;

  return (
    <div>
      {isLoggedIn ? (
        <h1>Welcome Back!</h1>
      ) : (
        <h1>Please Log In</h1>
      )}
    </div>
  );
}

export default App;
```

If `isLoggedIn` is `true`, React displays:

```
Welcome Back!
```

Otherwise, it displays:

```
Please Log In
```

Cont. ...

`if...else` vs Ternary

Using `if...else`:

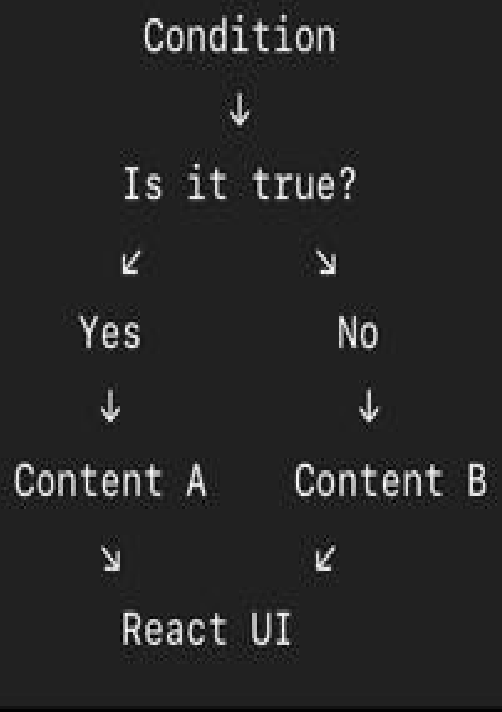
```javascript
if (isLoggedIn) {
  message = "Welcome";
} else {
  message = "Please Log In";
}
```

Using Ternary Operator:

```javascript
const message = isLoggedIn
  ? "Welcome"
  : "Please Log In";
```

The ternary operator is more concise for simple conditions.

How Does It Work?



React ES6 Template Strings

❖ **What are Template Strings?**

- Template Strings, also called **Template Literals**, are an ES6 feature that allows you to create strings using backticks **(``)** and easily insert variables or expressions using **${}**.
- In React, they are useful for creating **dynamic text**, **messages**, **URLs**, **class names**, and other strings.

❖ **Why are Template Strings Important in React?**

- Make string formatting easier.
- Allow variables to be inserted directly into strings.
- Reduce complicated string concatenation.
- Make dynamic content more readable.

Cont. ...

Basic Example

JavaScript

```javascript
const name = "Abel";

const message = `Hello, ${name}!`;

console.log(message);
```

Output:

```
Hello, Abel!
```

`${name}` is replaced with the value of the `name` variable.

Template Strings in React

JavaScript

```javascript
function App() {
  const name = "Abel";

  return (
    <h1>{`Welcome, ${name}!`}</h1>
  );
}

export default App;
```

The browser displays:

```
Welcome, Abel!
```

Cont. ...

Using Expressions

Template strings can also contain JavaScript expressions:

JavaScript

```
const price = 100;
const quantity = 3;

const total = `Total: ${price * quantity} Birr`;

console.log(total);
```

Output:

```
Total: 300 Birr
```

String Concatenation vs Template Strings

Traditional approach:

JavaScript

```
const name = "Abel";
const message = "Hello, " + name + "!";
```

Using Template Strings:

JavaScript

```
const name = "Abel";
const message = `Hello, ${name}!`;
```

The template string is generally **cleaner and easier to read.**


Thank you
GRATEFUL